

written. Thus, Numba enables the code to use our GPUs, if available. Notice that a JIT compiler is a type of compiler in which the compilation happens during execution of the code. As an example for Numba, consider the Python code shown in Figure 4. The same function called `fun()` is executed with and without using the Numba compiler. This is a simple function that finds the sum of the squares of integers from 1 to 9. Though this program is simple, on careful observation we can see that it has features that allow parallelization. From Figure 4, we can see that the code prints the same answer `405071317` on both occasions. However, we can also see that the execution time taken is different. Only half the time is taken to obtain an answer when the code is parallelized using Numba. Further, as the size of the problem being handled increases, the gap between the time taken by the parallelized and non-parallelized versions will also increase.

An introduction to SymPy

If we were discussing useful Python libraries, our discussion could have been about hundreds of such libraries. However, our discussion is about developing AI and machine learning based applications. Hence, I was a bit confused about the libraries to include in our discussion. After much deliberation, I have decided to briefly discuss SymPy. It is a Python library for symbolic mathematics. With an example, let us try to understand what symbolic mathematics is. Consider the program shown in Figure 5. It uses the function `Integral()` offered by SymPy to find the integral of ' $\sin(x) \cos(x) \tan(x)$ ' over x . Figure 5 also shows the output of this symbolic calculation. Notice that unlike the `integrate()` function offered by SciPy, which returns numerical results, the function `Integral()` provides exact

```
1 x = tf.constant([[1,2,3], [4,5,6], [7, 8, 9]])
2 print(x.get_shape())
3 y = tf.constant([1,1,1])
4 print(y.get_shape())
5 y = tf.expand_dims(y,1)
6 print(y.get_shape())
7 print(tf.linalg.matmul(x, y))

(3, 3)
(3,)
(3, 1)
tf.Tensor(
[[ 6]
 [15]
 [24]], shape=(3, 1), dtype=int32)
```

Figure 3: Matrix multiplication with TensorFlow

```
1 %time
2 fun()

CPU times: user 5 µs, sys: 1e+03 ns, total: 6 µs
Wall time: 10.5 µs
405071317

1 import numba

1 %time
2 @numba.jit
3 def fun():
4     y=0
5     for i in range(1,10):
6         x=i**i
7         y=y+x
8     print(y)
9 fun()

CPU times: user 2 µs, sys: 0 ns, total: 2 µs
Wall time: 4.77 µs
405071317
```

Figure 4: Parallelization with Numba

```
1 from sympy import *
2 x = symbols('x')
3 a = Integral(sin(x)*cos(x)*tan(x), x)
4 val = Eq(a, a.doit())
5 val
```

$$\int \sin(x) \cos(x) \tan(x) dx = \frac{x}{2} - \frac{\sin(x) \cos(x)}{2}$$

Figure 5: An example for symbolic calculation

symbolic results. SymPy is useful for performing statistical operations that have high relevance in the development of AI and machine learning based applications.

In the next article in this series, we will discuss Theano, an even more powerful tool for computing mathematical expressions. Theano is a Python library and optimising compiler for evaluating

mathematical expressions.

An introduction to probability

It is now time for us to have our first meeting with probability, yet another major topic of interest for AI and machine learning enthusiasts. A very detailed treatment of probability is beyond the scope of this series. I urge you to read the Wikipedia articles on 'Probability', 'Bayes' theorem', and 'Standard deviation' before continuing any further. These articles are simple and easy to follow. Some of the important terms and concepts like probability, independent events, mutually exclusive events, conditional probability, Bayes' theorem, mean, standard deviation, etc, are explained well in these articles and you will be able to follow our discussions on probability easily after reading them.

Let's begin with probability distributions. According to Wikipedia, 'a probability distribution is the mathematical

function that gives the probabilities of occurrence of different possible outcomes for an experiment'. Now, let us try to understand what a probability distribution function is. The most famous probability distribution function is the normal distribution, often also called Gaussian distribution (named after Carl Friedrich Gauss, one of the greatest mathematicians to live on this earth). If you plot the normal distribution, you